Development of BataviaAI: Neural Network-Based Racing Line Optimization and Pull-Point Prediction Using Heuristic and Shell Eco-marathon OBC Datasets

Batavia Gasoline Team
ID0030002
Universitas Negeri Jakarta

1. Introduction

Achieving energy efficiency in the transportation sector has become a global priority, driven by the need to minimize environmental impact and optimize resource utilization. The Shell Eco-marathon, an international competition aimed at pushing the boundaries of fuel efficiency, provides a unique platform for students and engineers to design, build, and operate ultra-efficient vehicles. This present study proposes a novel methodology for enhancing the performance of vehicles by employing artificial intelligence (AI) techniques and advanced mathematical methods to enhance energy efficiency, safety, and lap-time performance.

1.1. The Budget Problem

University-level teams participating in the Shell Eco-marathon face significant challenges in optimizing vehicle strategies due to limited access to tracks for physical testing. This limitation makes it difficult to gather comprehensive data needed to develop strategies, such as determining the optimal racing line or push-points for acceleration. Without a cost-effective alternative solutions, teams risk underutilizing the vehicle's potential for energy efficiency and competitive lap-times. This research addresses this gap by using GPS-based centerline data and AI-based models to create scalable, data-efficient solutions for trajectory analysis and performance optimization.

1.2. Complex Track Geometry

In the Shell Eco-marathon, the objective for competing vehicles are required to navigate a tracks with complex geometries, including straights, slopes, hills, and sharp corners. Achieving maximum energy savings while maintaining competitive lap-times necessitates the implementation of precise optimization strategies. Unlike conventional racing vehicles that operate continuously at optimal power, Shell Eco-marathon vehicles alternate between acceleration and gliding phases to minimize energy wastage. The transition point from acceleration to skidding is designated as the 'pull point'.

2. Related Work

Finding the optimal racing line and improving lap-time performance have long been areas of focus in both academic research and competitive racing. Cardamone proposed a genetic algorithm-based approach to optimize the racing line for racing games, striking a trade-off between minimizing curvature and distance (Cardamone et al., 2010). Their method decomposed tracks into segments and applied a simulation-based evaluation, demonstrating the power of evolutionary algorithms in trajectory planning. In real-world applications, racing line optimized through quadratic programming, balancing shortest path (SP) and minimum curvature path (MCP) to improve stability and speed, but lacked adaptability to varying track conditions. Building on this, our research integrates AI-driven adaptive strategies, skewed probability distributions, and safety constraints to enable dynamic adaptation to real-time conditions. This scalable approach is suitable for university teams with limited resources, bridging simulation and practical application.

3. Methodology

- Determining the optimal racing line using GPS data and skewed probability distributions to identify energy-efficient paths on the track.
- Calculating the most efficient push-points and corresponding acceleration durations and energy dynamics.
- Implementing safety constraints, such as limiting maximum speed during turns, and proper instrumentation to prevent vehicle instability or roll-over.
- Creating a device and method to help driver being able to apply analysis accurately on track to increase safety by helping driver psychologically.

4. Track Definition

In this section, the terminology and notation employed throughout the remainder of this paper are defined. Firstly, the representation of the track is described. Thereafter, we introduce the concepts of the centre-line, centre-points, and track nodes. Finally, the methodology for calculating the optimal lap-time is outlined.

4.1. Track Representation

To define the racing line, it is essential to first establish the representation of the track. Most AI methods utilize multiple nodes placed around the track, with algorithms such as Dijkstra or A* (A-star) employed to determine the optimal racing line. In this

research, we also employ nodes to identify the most optimal racing line.

This method aligns with the raw sensor data from the Shell Eco-marathon, demonstrated by the time series signal using track coordinates. Nodes determined by the track's centerline are utilized. Shown below as an example data from the Shell Eco-marathon Asia Pacific and the Middle East 2024, gathered by Si Jayaraya 2.0, our 2024 Simulate to Innovate award-winning gasoline urban concept vehicle.

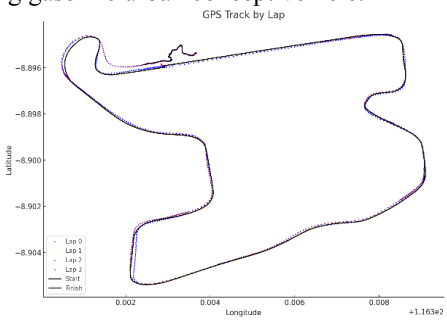

Figure 4.1 GPS Data Track by Lap Provided by Shell Eco-marathon in 2024, visualized using Matplotlib.

5. Building Heuristic Method Dataset

5.1. Obtaining Track Data

The track walls by using the width values at each point along the center line, which must be smooth with consistent point spacing. These points are connected to form the track wall. The present study focuses on the Jakarta International E-Prix Circuit (JIEC) in Jakarta, Indonesia.

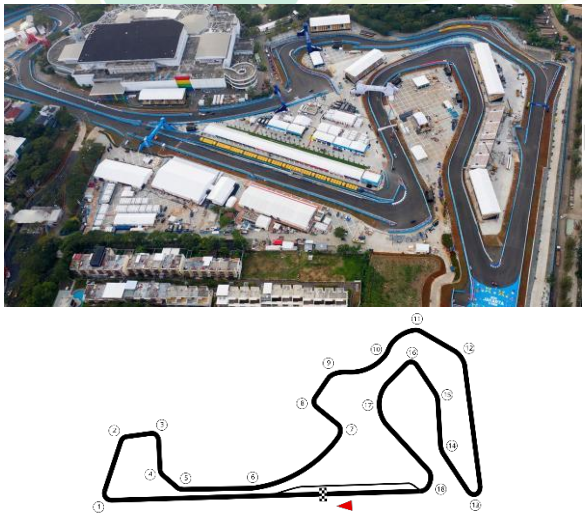

Figure 5.1 Jakarta International E-prix Circuit (JIEC), Jakarta, Indonesia

The track points will be gathered and connected to form a track center line.

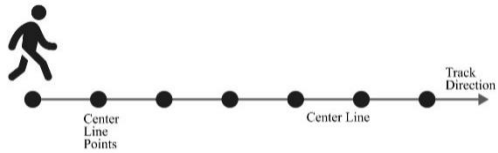

Figure 5.2 Track Walk Method

Track Walk Method (Fig 5.2 & 5.3). We began by gathering data through track walk sessions. During these sessions, the analyst walked around the track, recording data using an IoT-based data logger equipped with a U-blox Neo-6M module. This time-series data was collected and recorded live in InfluxDB.

Figure 5.3 Gathering data during track walk at JIEC with NEO-6M GPS tracker installed.

Google Earth Method (Fig 5.4). Another cost-effective and time-saving method involves utilizing track point data. Google Earth employs the KML (Keyhole Markup Language) file format, which contains nearly all the necessary information to generate an optimal racing line. KML files use XML to represent geographic features such as points, lines, polygons, and images. They can also include custom data, such as names, descriptions, and styles, to enhance the visualization of geographic information.

Figure 5.4 Jakarta International E-prix Circuit Center Line Traced with Google Earth

The utilization of Google Earth offers significant advantages in the efficient storage and management of multiple track data, optimizing both time and energy resources. The track maps generated within Google Earth can be exported in the Keyhole Markup Language (KML) format. Each track within the KML file is associated with a unique placemark identifier. For instance, in our KML file, the placemark identifier for the Jakarta International Expo Center (JIEC) is represented as follows:

`Placemark id="05A6834C2435BA23DD63"`.

Subsequently, this placemark can be referenced within the KML file to extract the corresponding data sequence for conversion into a Comma-Separated Values (CSV) format. In this context, the extracted data is organized into two primary fields: t-lat (latitude) and t-long (longitude). This systematic approach facilitates the effective analysis and utilization of geospatial data derived from Google Earth.

5.2. Cleaning Track Data

Unfortunately, the collected data lacks the necessary smoothness to generate an accurate race track. Therefore, it is essential to smooth the distances between each data point.

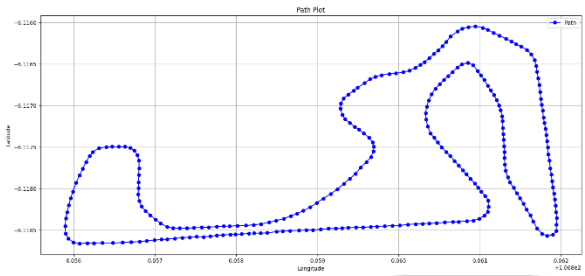

Figure 5.5 Raw data with distance inconsistency between each points.

To achieve a smooth data line representation, two main methods can be used: the `scipy.interpolate` method and the Savitzky-Golay filter. The `scipy.interpolate` method is particularly suitable for tracks characterized by complex twists and turns, as it provides flexible smoothing and allows a parametric representation that is useful for subsequent processing, such as calculating derivatives.

In contrast, the Savitzky-Golay filter, implemented as `savgol_filter` of the `scipy.signal` module, would be better if the data is already relatively smooth and a fast smoothing technique is desired. In our analysis, we chose the Savitzky-Golay method due to time constraints. Our findings showed that the `scipy.interpolate` method unintentionally smoothed the straight segments of the trajectory in a way that resembled a parabolic curve, which did not suit our specific needs. Therefore, the Savitzky-Golay filter was deemed a more appropriate choice for our smoothing needs.

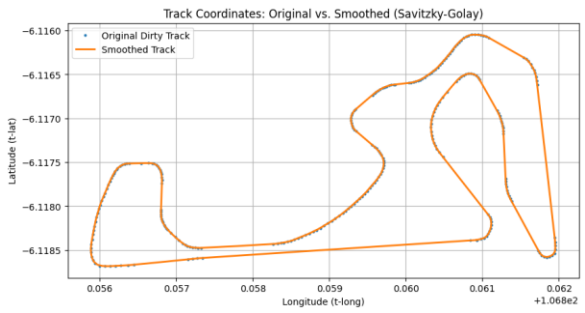

Figure 5.6 Smoothed Circuit Line (Orange)

5.3. Track Final Form

Once the smoothed version of the track is obtained, the track wall can be generated by utilizing the width value of the track.

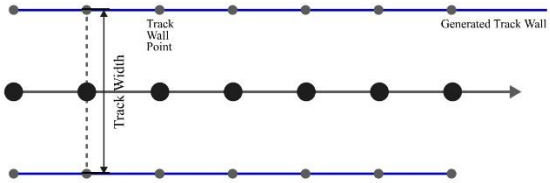

Figure 5.7 Generated track wall based on the center line.

To describe the useful way of segmenting each track, it is necessary to set up centerline points that match the geometry of the track. In this context, the lateral adjustment option is provided in the code so that the user can change the position of the centerline, either pulling it into the track or pushing it out. The calculation begins with determining the direction vector for the track.

For each segment, the differences in longitude (dx) and latitude (dy) between consecutive points are calculated. The magnitude of the direction vector is then determined using the Euclidean distance formula, and it is normalized to produce a unit vector..

After computing new points, a Pandas DataFrame is used to combine the original smoothed points with the newly calculated ones. In the end, all the generated points are saved to a new CSV file for further analysis and use.

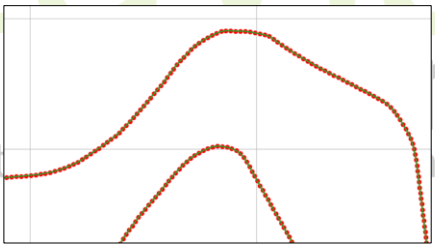

Figure 5.8 Track Center Line Points Generated

The track's segmentation is derived from the smoothed centerline points. The code initiates by importing the CSV file into a Pandas DataFrame for structured data manipulation. It then iterates through each point in the track, commencing from the second point, to compute the direction vector between consecutive points by calculating the differences in longitude and latitude and normalizing the vector using the Euclidean distance formula.

Furthermore, the code establishes an empty list designated for segmentation lines and defines the number of segments n to be generated between the inner and outer walls. The code generates linearly spaced points between the inner and outer wall points for each segment and appends them to the 'segmentation_lines' list. This approach effectively represents the geometry of the track through segmentation.

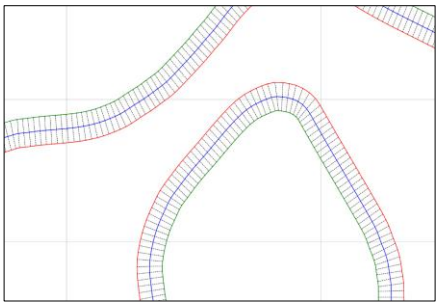

Figure 5.9 Generated Track Segmentation

We acknowledge that the artificial intelligence (AI) system requires extensive training on a substantial dataset of track data. Currently, our dataset comprises information from 35 distinct racetracks. To mitigate the limitations by this relatively small dataset, we have implemented innovative techniques such as scaling, flipping, and reversing the tracks. Through these methods, we are able to generate up to 2,549 unique racetrack configurations, thereby significantly enhancing the diversity of the training data available for the AI system.

The code for this method is available: https://github.com/bataviateam2/data-telemetry-2025/blob/main/Building-Dataset.ipynb

Table 5.1 Track Dataset

Dataset	Real	Augmented	Total
Training Data	28	2204	2232
Validation Data	3	226	229
Testing Data	4	119	123
Total	**35**	**2549**	**2584**

5.4. Generating Racing Line with Heuristic Method

For every center line points along the track we put multiple nodes along the perpendicular line across the center line points. This perpendicular line become the x axis of the probability distribution graph while the y axis $f(x)$ as the probability density. The probability distribution use a skewed Gaussian Distribution defined as:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{(x-\mu)^2}{2\sigma^2}}$$

The problem with Gaussian distribution is when it is skewed to the right side, it will create an infinite number of candidates along the x axis. Since the probability distribution draw as a curve, there's a potential where the nodes will be dense near it's center. Which has a possibility of staying around the highest region of the curve and not giving an exact nodes.

The solution of this is by creating more skewed and simpler approach to the probability density. This solution will have much simpler form.

$$f = ax + b$$

The shape that formed from this density function has more spesific points. This probability density has much wider range of possibilities being tried and has more freedom to jump toward either of the two sides.

Safety Factor. On the other hand, we also determine the safety factors. There's a potential condition where the vehicle could be rolled-over during the lap or when taking turns. This could be avoided not only by the vehicle design but also by limiting the velocity during the turn. Let v_{max} as the maximum velocity that the vehicle could handle. This safety measures will be part of our key constraints of the lap.

$$v(s) \leq v_{max}(s)$$

This indicates that the turning radius of the car cannot be too small.

$$r \geq r_{min} \;\&\; i.e. k \leq k_{max}$$

This maximum speed limit will also help limiting the AI on exploring throughout the iteration. Speed limit by the engine or by regulations. The car cannot run with infinitely large speed and there is a maximum speed limit.

Push-point. Push-point is another variable that have to be considered in managing the eco-car efficiency performance during the lap. Push-point is the points of the track taken from the any single point of racing line where the car is turning on the powertrain to accelerate. This push-point determined by the calculation of the most optimal lap time. The key constraints of the push-point is determined by the energy dynamics on the track. Energy dynamics is an incorporated the energy conservation strategy:

$$E_{motor} + E_{glide} \leq E_{total}$$

Where the E_{motor} is energy used by the motor/engine/powertrain and E_{glide} is the energy saved during gliding.

The most efficient racing line in this algorithm is one that requires the least energy and has the smallest curvature. Less curvature, in the absence of one centripetal force, results in lower cornering efficiency. The two primary components of this algorithm are the determination of the optimal racing line and the calculation of velocity. All three are equally crucial.

We have developed rules that dictate the curvature of the racing line to ensure it remains as smooth as possible and achieve the optimal energy trajectory. Each point n on the track is evaluated by the algorithm for degree of bending, starting with $(n-1)$ and ending with $(n+1)$. In accordance with the rule, if the angle formed by the segment from $(n-1)$ to n is sharp, then the next points $(n+1)$ must have an even more gentle curve.

Dream, Design, Create For Better Future Environment

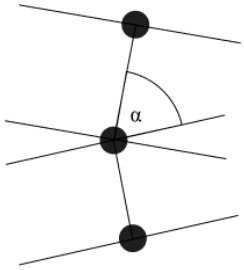

Figure 5.10 The racing line bend between 3 points.

Using this method require iterating the algorithm up to 40,000 times to achieve an optimal racing line. Otherwise, the resulting lines may appear fragmented and suboptimal. This extensive process of generating the racing line is not efficient in terms of time and energy consumption. Because of this problem we have developed an artificial intelligence system to enhance the efficiency and speed of this process, enabling the generation of optimal racing lines in more efficient process.

5.5. Vehicle Dynamics Dataset

Upon generating the racing line, the algorithm calculates various variables related to its conditions, aggregating data for the training of an AI model. The data is divided into two types: individual data points, which include variables such as drag coefficient, air density, frontal area, rolling resistance, vehicle mass, gravitational acceleration, engine power, and turning radius; and time-series data, which vary along the track and encompass metrics such as fuel consumption and velocity. This structured approach ensures that the AI has access to comprehensive and relevant data for effective learning and analysis.

5.6. Heuristic Dataset Result (1st Dataset)

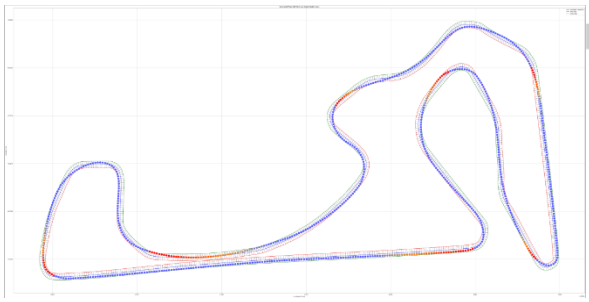

Figure 5.11 Heuristic Method Result

Figure 5.11 illustrates the outcomes of the heuristic method employed to generate the racing line. The variable t is denotes as the target lap time, and the algorithm calculates the average velocity required to complete the track by sequentially analyzing the vehicle's position and the corresponding velocity needed along the track. The analysis considers the phenomenon of engine disengagement in the context of gliding at specific points in the race. After determining the required velocity, the algorithm undertakes an iterative calculation of energy expenditure in order to identify optimal acceleration points, or "pull-points". This refinement of the racing line is intended to improve performance.

Table 5.2 Generated Dataset Examples after 3 runs.

Run	Iteration	Track Time (m:s:ms)	Est. Energy (J)
1	35,000	25:00:911	591.66
2	40,000	25:15:434	581.76
3	45,000	24:45:499	576.54

The generation of this dataset required our team an entire month, as it involved significant time and resource investment. Subsequently, this dataset was utilized to train the artificial intelligence (AI) system.

5.7. Shell Eco-marathon OBC Dataset (2nd Dataset)

Shell Eco-marathon OBC data contains multiple variables. But not all variables available every year, so we only took the ones that only available on all 3 years. We took data from our Urban Concept vehicle data, Si Jayaraya and Si Jayaraya 2.0. This data gathered from Shell Eco-marathon APME 2022 to 2024. This available data are:

Name	Description
rec_id	Unique record identifier.
gps_latitude	Latitude data for GPS coordinates.
gps_longitude	Longitude data for GPS coordinates.
gps_speed	Speed data from GPS.
lfm_temperature	Temperature data from the LFM sensor.
lfm_integratedcorrflow	Integrated flow data from LFM.
lfm_instantflow	Instantaneous flow data from LFM.
gfs_gasflow	Gas flow data from GFS.
gfs_gastemp	Gas temperature data from GFS.
gfs_netgasvolume	Net gas volume data from GFS.
jm3_netjoule	Net joule data from JM3.
jm3_voltage	Voltage data from JM3.
jm3_current	Current data from JM3.
message_id	Identifier for messages.
volt_voltage	Voltage data.
volt_difference	Difference in voltage.

These are dataset that available on all 3 years. We can use this data for training. To train the model, we took one part of sequence n and took 2 steps forward $(n+1)$; $(n+2)$ and backward $(n-1)$; $(n-2)$ of the data. This means there's an exception for the 2 first data and 2 last data because we can't generate the next and previous from both.

Data 1
Data 2
Data 3
Data 4

Data 5
Data 6
Data 7
Data 8

Figure 5.12 Data taken for training (Green), data not taken for training (red).

6. Neural Network

Two methodologies utilizing neural networks will be employed in this research. The first method involves generating the racing line and identifying pull-points based on the heuristic method data previously developed. The second method focuses on training the artificial intelligence (AI) using actual track data provided by the Shell Eco-marathon's On-Board Computer (OBC). However, the actual track data is limited in size and may be less accurate than heuristic data, potentially affecting AI training and model predictions.

6.1. Training from Heuristic Method Dataset

Training the artificial neural network (ANN) started with defining the problem: predicting the optimal racing line, identifying pull-points, and evaluating performance metrics such as energy efficiency and lap time. The AI calculates predictions for each track segment using two sets of inputs: static parameters (vehicle mass, estimated time, power, rolling resistance and drag coefficient) and time-series data (previous position, angle and speed).

In terms of that, the heuristic methods are based on mathematical relationships and assumptions based upon physical models or past observations like drag equation or power loss approximations.

The ANN architecture includes an input layer (integrating the static and time-series parameters), one or more hidden layers, and an output layer. The output layer iteratively generates three predictions as the vehicle navigates the course. During training, the sigmoid function is used to compute gradients and predict normalized track positions (0 to 1). Pull point classification also uses the sigmoid function to label points as pull points (1) or not (0). Although more computationally expensive than the Rectified Linear Unit (ReLU), the sigmoid function is preferred due to its probabilistic output within a limited range.

The AI monitors speed to identify points of attraction. When the lowest speed is detected, the AI moves towards the attraction point and stops accelerating when the highest speed is reached. This process is shown in Figure 5.12.

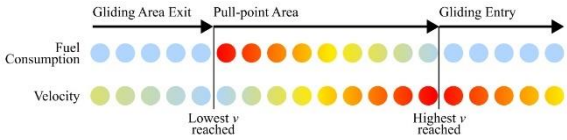

Figure 6.1 Pull-point Area

Due to the limitations of the artificial intelligence (AI) system in calculating fuel consumption, we have implemented an additional heuristic method as a supplementary layer following the output of the neural network. By analyzing the predicted velocity, the AI can ascertain whether the powertrain is engaged or in a coasting state. This approach enhances the model's capability to estimate fuel consumption indirectly, thereby improving the overall accuracy of performance predictions.

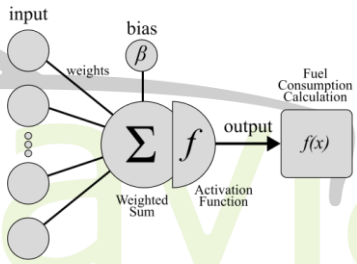

Figure 6.2 Overall AI Diagram

Inputs are:

Table 6.1 Neural Network Imput

Name	Description
vhc-mass	Vehicle Mass
est-time	Estimated/Target Time
vhc-power	Engine/Motor Power
roll-res	Rolling Resistance
vhc-drag	Vehicle drag coefficient
l	Position along the perpendicular line
a	Radius between 3 points
v	Vehicle velocity

Dream, Design, Create For Better Future Environment

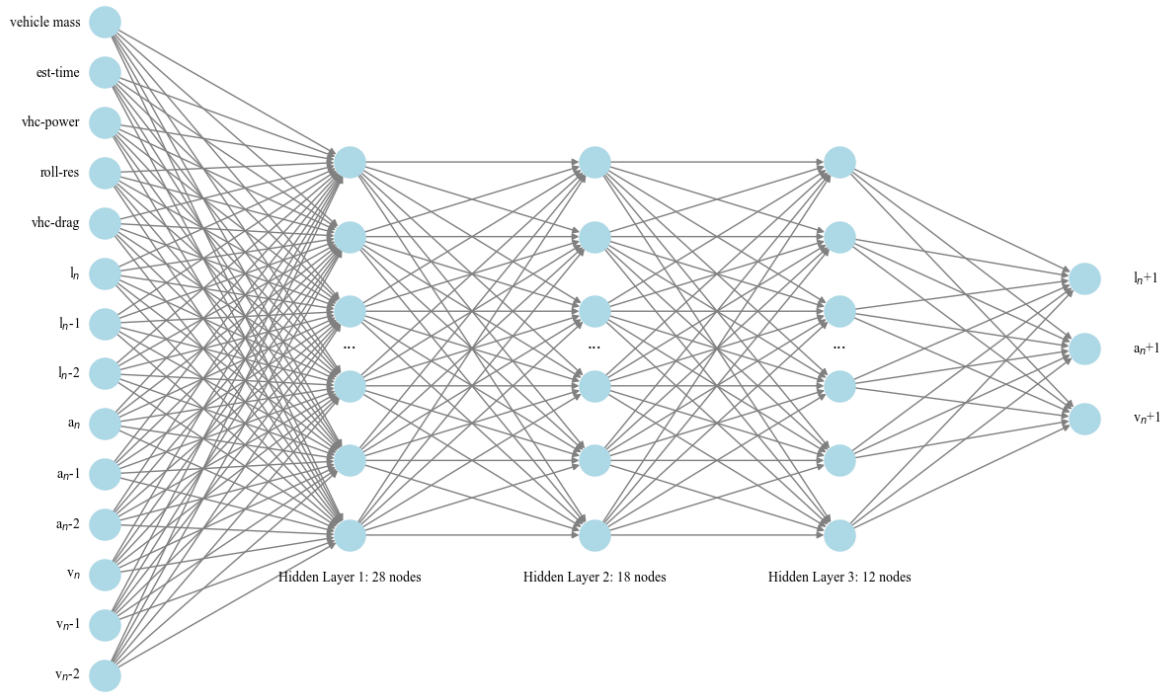

Figure 6.3 Network of AI using Heuristic Method Dataset.

6.2. Training from Shell Eco-marathon Data Telemetry

Another alternative approach is to use the data available from the Shell Eco-marathon On-Board Computer (OBC). In a field dominated by Large Language Models (LLM) and Artificial Intelligence (AI), short-term memory networks (LSTM) tend to be used in conversation. This method works particularly well for sequential data; it can be used in interconnected areas as seen in speech recognition and natural language processing. Implementing such an architecture would perfectly represent the sequential data collected from the OBC at the Shell Eco-marathon. By integrating the LSTM with the OBC data, the model overcomes the drawbacks posed by missing gradients and is able to retain memory over a long period of time.

The main disadvantage in using Long Short-Term Memory (LSTM) networks lies in its relatively slower processing speed compared to Transformer architectures, especially when the size of the data set increases or when handling long data sets. Although LSTM showed commendable relevance with the Shell Eco-marathon (SEM) telemetry data, its performance could be hampered by this limitation.

Transformers, originally developed for natural language processing (NLP) tasks, have gained significant attention in the domains of time series analysis and sequence modeling due to their parallel processing capabilities and effectiveness in capturing complex relationships in data.

The transformer's attention mechanism is designed to solve parameter dependencies (e.g., power and speed and even drag) of different orders i.e., non-linear dependency structure in telemetry data where foresight predictions are more dependent on these parameters.

Central to the Transformer architecture is the "self-attention mechanism," which enables the model to selectively focus on specific segments of the input sequence. In the context of SEM telemetry, this allows the model to attend to particular timestamps or sections of the racetrack, thereby enhancing its analytical capabilities.

Simply put, ChatGPT is an LLM that uses transformer architecture to generate the next word in a text. For Batavia Team, we utilize it to generate the next point for the racing line, repeating the process for several laps to determine the most optimal racing line..

SEM telemetry data often involves complex relationships between points that are far apart, such as the influence of a vehicle's behavior at one corner on its performance a few meters later. Transformers is particularly adept at capturing these dependencies, effectively mitigating the missing gradient problem that artificial neural networks (RNNs) and even LSTMs may face. These characteristics position Transformers as a more suitable choice for modeling the inherent complexity of telemetry data SEM.

Table 6.2 Comparison between LSTMs and Transformers

Feature	LSTMs	Transformers
Speed	Slower (Sequential Processing)	Faster (Parallel Processing)
Long Dependencies	Limited	Excellent
Scalability	Moderate	High

Batavia team
Dream, Design, Create For Better Future Environment

Vanishing Gradient	Handled Well	Not an issue
Multivariate Data	Supported but less flexible	Supported and flexible

The input layer of the transformer process is sequential data. Instead of inputting all the inputs at once, we create a sequence of time steps for each feature. This input feature uses the same way as the heuristic method:

$$v_{n-2}, v_{n-1}, v_n, a_{n-2}, a_{n-1}, a_n, l_{n-2}, l_{n-1}. l_n$$

This sequence length is 3 for each variable. The single data known as contextual or static inputs:

vhc-drag, roll-res, vhc-power, est-time, vhc-mass

For the transformer, the input features are the sequence features and the static inputs. The sequence features (*v,a,l*) are structured as a time-series matrix with a shape (Batch Size, Sequence Length, Feature Dimensions). The static inputs are the 5 contextual features that are appended to each time step, giving a final input shape of: (Batch size, 3, 8) which is 3 features and 5 contextual inputs.

The transformer encoder processes the sequence data. 2-4 encoder layers used for moderately complex sequence bags and 4-8 attention heads (depending on model complexity and dataset availability. For the hidden size, it should be divisible by the number of attention heads, in this case we use the range 16-64.

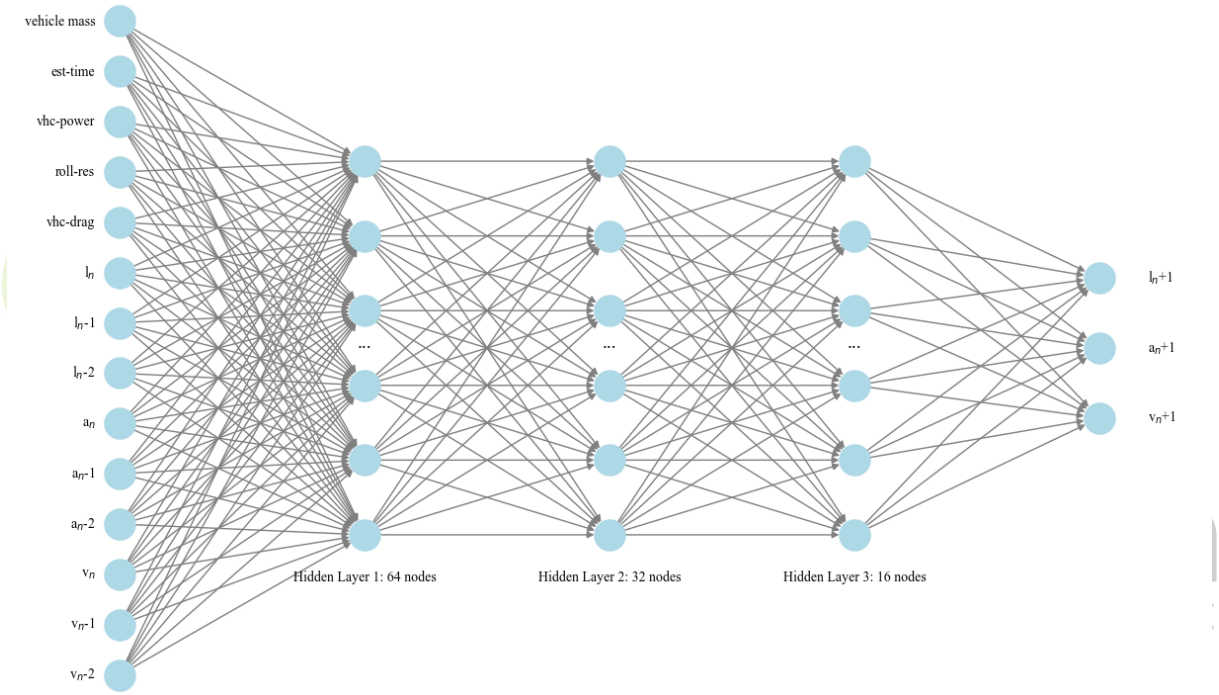

Figure 6.4 Network of AI using SEM' OBC Method Dataset

7. Result

In comparison, both neural network method have their own characteristics:

Table 7.1 Comparison using Heuristic Data and OBC's Data

Aspect	Heuristic Data	OBC's Data
Data Complexity	Simplified, rule-based	Complex, real-world
Information Density	Lower	Higher
Hidden Layers	28, 18, 12	64, 32, 16
Model Focus	Efficiency and generalization	Detailed pattern extraction

Because heuristic data has simpler methods, consistent methods and consistent results compared to SEM OBC/real-world data. This happens due to various conditions on the track. Things like DNFs, wind direction, and other track conditions can greatly affect the results even after data cleaning.

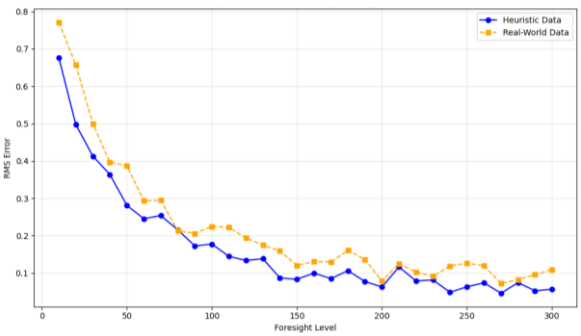

Figure 7.1 The impact of foresight on the accuracy of predicting the racing line.

Foresight determines how many points on the perpendicular line are taken into account in each analysis window (Fig. 7.1). Having a better view of the

previous and future track layouts will result in more accurate predictions of the racing line. However, increasing Foresight also increases the input size for the artificial neural network (ANN), thus making the problem more complex.

Using K-fold cross-validation helps us to evaluate on multiple data splits, mitigating the model overfitting to a single subset of telemetry data and making a more reliable estimation of prediction accuracy. The decrease of RMS error in time is the model learning temporality in telemetry data. As the network sees more and more training examples, it generalizes better to the relations between features and ordered outputs.

The error or offset between the manually drawn racing line and the racing line generated by both methods was calculated. The error is obtained by looking at how large the offset is from the manually drawn racing line. Then, the average error across the entire track was calculated.

Table 7.2 Error of Heuristic Dataset Trained AI

Circuit	Error	AI Run-Time
Jakarta E-prix	0.898	02:14:125
Best-Friend Circuit	0.342	01:24:177
Mandalika	1.213	02:55:985
Lusail (SEM-layout	1.453	03:34:212

For smaller tracks like the Jakarta International E-prix Circuit, the AI tends to have a lower number of errors. While for wide and long circuits like Lusail and Mandalika, the AI tends to explore more positions along the track. It cannot be stated whether AI-written data or manually-written data is more precise or better unless testing it on an actual track. These error numbers only show how much they differ from our manually predicted racing lines.

Table 7.3 Error of OBC's Dataset Trained AI

Circuit	Error	AI Run-Time
Jakarta E-prix	1.298	01:32:125
Best-Friend Circuit	1.122	01:14:177
Mandalika	1.937	02:45:543
Lusail (SEM-layout	2.295	04:12:441

Data generated by the AI struggles on wider trajectories because it has a longer time. Because the wider the trajectory, the wider the number of probabilities for each segment. The error rate of the OBC-trained AI is still very high. This is due to the lack of track data and the variety of track conditions. The racing lines produced by the two methods look very similar. However, their analysis of speed is different. This affects their difference in providing vehicle pull points.

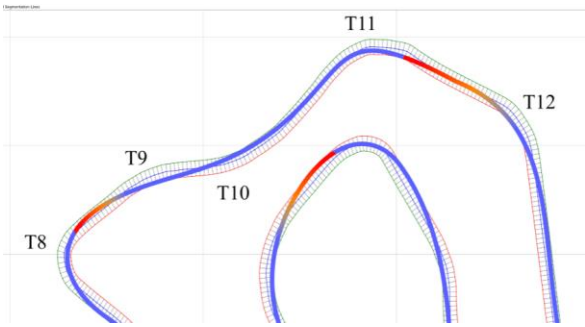

Figure 7.2 Racing line generated by AI Trained by Heuristic Dataset

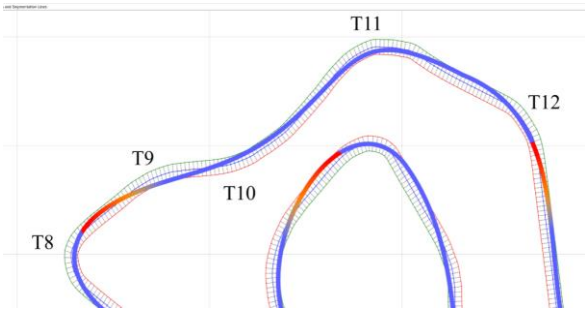

Figure 7.3 Racing line generated by AI Trained by OBC Dataset

The colors on the racing line visualize the fuel consumption on the track. The AI trained with heuristic data shows a short pull from turn 8 and a longer pull from turn 11 before the straight after turn 12. In contrast, the OBC-trained data set shows a longer pull from turn 8, coasting into turn 12, followed by a short pull after turn 12. Despite this difference in strategy, the two approaches produce similar time estimates.

8. Race Strategy and Driver's Performance

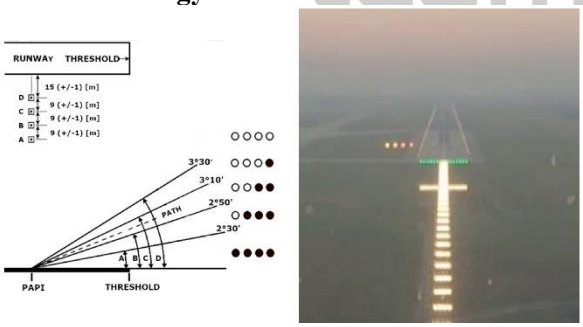

Figure 8.1 Aircraft PAPI Lights

To improve the driver's decision-making on the track, we have transferred the generated racing line coordinates to the ESP32 module. An easy-to-use and user-friendly interface using an I2C LCD has been developed, which can be mounted on the steering wheel to provide real-time feedback to the driver regarding acceleration and track position.

The first line of the display functions similarly to the Precision Approach Path Indicator (PAPI) lights used in aviation, assisting the driver in determining whether they need to adjust their position to the left or right. The second line indicates when the driver should

accelerate. This two-line interface is designed to be intuitive and easy to understand, making it particularly suitable for drivers operating under high-pressure mental states. By facilitating clear communication of important information, the system aims to improve safety from a psychological perspective.

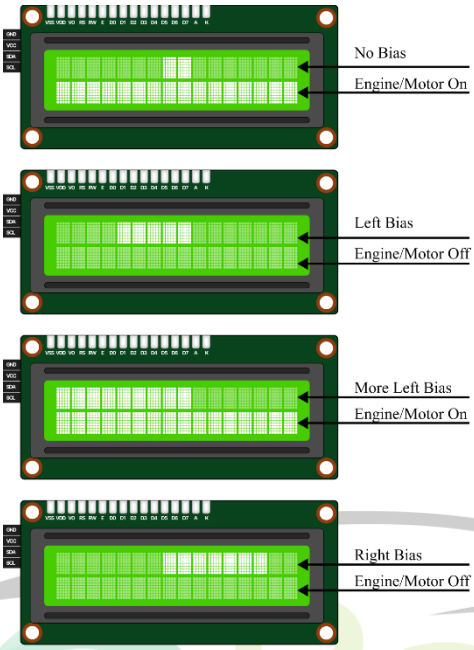

Figure 8.2 i2C LCD Different Information

This method still has a major drawback due to the lack of accuracy of GPS data. The better the device we have, the more accurate the results we will get. Even so, after we did several test drives, we still got good and constant results with the same strategy. The test used our Prototype Gasoline vehicle, Si Pitung. The improvement parameters were based on the results of our first trial at JIEC, which was 555.32 km/l.

The conclusion from the feedback given by the driver is that the AI trained with heuristic data gives more accurate answers compared to the AI trained with OBC dataset. Sometimes the AI suggests pull-point even though the vehicle speed is still not slow enough, our analysis says that the data generated with the heuristic data is more than the data trained with the OBC dataset, which affects the results given. The OBC dataset has a wider range of conditions compared to the heuristic dataset. AI trained with OBC datasets requires more datasets to get more accurate results. Because things like wind direction, for example, are not given in the OBC CSV document. The situation on the track should be considered more to train this AI..

Table 8.1 Si Pitung Results after 9 different runs with 3 different strategies at Jakarta International E-prix Circuit.

Attempt	Km/L	JM (J)	Lap-time	Improvement	Driver Feedback
Strategy 1 (S1 / No AI)					
1	549.15	3.4	25:43:003	-1.05%	Unstable and based on manual strategy. No clue of where the improvements or changes should be placed.
2	527.05	3.5	25:15:434	-5.85%	
3	554.76	1.5	25:00:911	-0.04%	
Strategy 2 (S2 / AI – Heuristic Dataset)					
4	867.1	1.2	25:12:453	+56.23%	More stable information given by the AI. Helps on determining pull-point, but not always right or accurate.
5	912.2	1.6	25:04:628	+64.36%	
6	887.4	1.3	25:00:455	+59.89%	
Strategy 3 (S3 / AI – OBC's Dataset)					
7	667.32	2.0	25:12:113	+20.24%	More unstable information in every attempt compared to S2
8	546.45	2.5	25:23:147	-1.54%	
9	647.14	2.3	25:07:235	+16.60%	

9. Room of Improvement

The main difficulty in generating racing lines is the lack of datasets from SEM OBC, if we take up to 6-7 years of data on the same category, it will help improve the SEM OBC trained AI to produce more accurate results. Another improvement is to combine the two methods between the heuristic dataset and the OBC dataset. To improve both methods, we can use LIDAR data generated by cars traveling around the track, scan the actual track conditions, and map them. By doing this, we can generate more accurate datasets and results.

References

Cardamone, L., Loiacono, D., Lanzi, P. L., & Bardelli, A. Pietro. (2010). Searching for the optimal racing line using genetic algorithms. *Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games*, 388–394. https://doi.org/10.1109/ITW.2010.5593330